

Introducción a la complejidad computacional

Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Agosto 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Contar, línea a línea, cuántas veces se ejecuta cada instrucción de un programa en C.
- Expresar el total de instrucciones como una función T del tamaño de la entrada.
- Distinguir el mejor y el peor caso de un algoritmo y decir cuál se reporta.
- Comparar dos algoritmos correctos por su número de instrucciones, sin usar el reloj.

Objetivos de aprendizaje II

Objetivos de Aprendizaje del curso involucrados

- OA1: apropiar conocimientos de computación mediante el diseño e implementación de soluciones a problemas pequeños.
- OA3: describir ideas con argumentos precisos y basados en evidencia.

Referencia bibliográfica

T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.^a ed., sección 2.2 (*Analyzing algorithms*); R. Thareja, *Data Structures Using C*, capítulo 2.

Contenido I

- 1 ¿Cuál algoritmo es mejor?
- 2 Contar instrucciones línea a línea
- 3 Mejor y peor caso
- 4 Cuando el conteo crece más rápido
- 5 Ejercicios
- 6 Resumen y referencias
- 7 Pistas de los ejercicios

Contenido I

- 1 ¿Cuál algoritmo es mejor?
- 2 Contar instrucciones línea a línea
- 3 Mejor y peor caso
- 4 Cuando el conteo crece más rápido
- 5 Ejercicios
- 6 Resumen y referencias
- 7 Pistas de los ejercicios

Lo que quedó de la clase pasada I

- Un programa en C pasa por cuatro etapas antes de ejecutarse; gcc las automatiza.
- `-Wall` y `-Wextra` avisan de problemas que compilan pero fallan al correr.
- En el paradigma imperativo la variable es un espacio de memoria que cambia de estado, instrucción por instrucción.

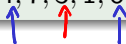
Hoy esas instrucciones se cuentan.

Un problema I

Dado un arreglo de n enteros, contar cuántos valores pares están en posiciones pares.

Ejemplo

Para el arreglo $\{4, 7, 3, 1, 8, 5\}$ la respuesta es 2.



¿Entendimos el problema? I

- **Entradas:** un arreglo de n enteros.
- **Salida:** cuántas posiciones pares guardan un valor par.

Traza sobre $\{4, 7, 3, 1, 8, 5\}$

Posición	¿Posición par?	Valor	¿Valor par?
0	sí	4	sí
1	no	7	
2	sí	3	no
3	no	1	
4	sí	8	sí
5	no	5	

Cuentan las posiciones 0 y 4: la respuesta es 2.

Primera solución I

Revisar todas las posiciones y preguntar por las dos condiciones.

```
int contar_v1(int datos[], int n) {  
    int cuenta = 0;  
    int i = 0; 0 — n-1  
    while (i < n) {  
        if (i % 2 == 0 && datos[i] % 2 == 0) {  
            cuenta = cuenta + 1;  
        }  
        i = i + 1; 0 1 2 3 ... n-1  
    }  
    return cuenta;  
}
```

Segunda solución I

Visitar solo las posiciones pares, avanzando de dos en dos.

```
int contar_v2(int datos[], int n) {  
    int cuenta = 0;  
    int i = 0;  
    while (i < n) {  
        if (datos[i] % 2 == 0) {  
            cuenta = cuenta + 1;  
        }  
        i = i + 2;  
    }  
    return cuenta;  
}
```

0 - 2 - 4 - ... - n

Sobre {4, 7, 3, 1, 8, 5} ambas responden 2.

¿Cuál de las dos es mejor? I

Las dos son correctas. Para escoger hace falta un criterio.

- ¿La que tiene menos líneas?
 - ¿La que corre más rápido?
 - ¿La que usa menos memoria?
- complejidad computacional
complejidad espacial

En parejas: escoja uno de los tres criterios y escriba en una frase cómo lo mediría.

{

Los tres criterios, uno por uno I

La longitud del código no dice nada: ambas tienen casi las mismas líneas. Quedan dos recursos por examinar: el tiempo de procesamiento y la memoria.

Primer intento: el reloj I

Para probar el reloj se toma un programa pequeño: sumar los enteros de 0 a $m - 1$.

```
long suma_hasta(long m) {  
    long i = 0;  
    long suma = 0;  
    while (i < m) {  
        suma = suma + i;  
        i = i + 1;  
    }  
    return suma;  
}
```


$$\sum_{i=0}^n i = \frac{n(n-1)}{2}$$

Primer intento: el reloj II

Tiempos medidos con `time` en un mismo computador

m	Tiempo
10^7	0.012 s
10^8	0.118 s
10^9	1.141 s

Cada vez que m se multiplica por 10, el tiempo se multiplica por cerca de 10. El reloj sí cuenta algo.

El reloj engaña I

El mismo programa, el mismo computador, la misma entrada
 $m = 10^9$:



Corrida	Tiempo
Primera	1.418 s
Segunda	0.958 s
Tercera	0.948 s
Compilado con gcc -O2	0.266 s

El tiempo no mide el algoritmo

El tiempo de ejecución depende del computador, de la carga del sistema y del compilador. Mide una corrida, no el algoritmo. Se necesita una medida que dé lo mismo en cualquier máquina.

La salida: contar instrucciones I

Definición

La complejidad computacional es una medida de la cantidad de recursos que un algoritmo usa para resolver un problema. Los dos recursos principales son el procesamiento y la memoria.

El modelo de conteo (CLRS, sección 2.2)

Cada instrucción simple (una asignación, una comparación, una operación aritmética) cuesta lo mismo y se cuenta como una unidad. El costo de un algoritmo es el número de instrucciones que ejecuta, expresado como función del tamaño de la entrada.

Ese número es idéntico en un portátil viejo y en un servidor nuevo: solo depende del algoritmo y de la entrada.

Contenido I

- 1 ¿Cuál algoritmo es mejor?
- 2 Contar instrucciones línea a línea
- 3 Mejor y peor caso
- 4 Cuando el conteo crece más rápido
- 5 Ejercicios
- 6 Resumen y referencias
- 7 Pistas de los ejercicios

El método I

Al frente de cada línea se anota cuántas veces se ejecuta. El total, escrito como función T del tamaño de la entrada, es el costo del algoritmo.

El caso más simple I

Calcular el doble de un entero.

Línea	Veces
<code>int doble(int n) {</code>	
<code> int resultado = 2 * n;</code>	1
<code> return resultado;</code>	1
<code>}</code>	

Las llaves no ejecutan nada. El total es

$$\underline{T(n)} = 1 + 1 = 2.$$

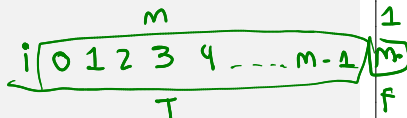
Costo constante

`doble(4)` y `doble(1000)` ejecutan las mismas 2 instrucciones.
Cuando T no depende de la entrada, el costo es constante.

¿Y si hay un ciclo? I

Sumar los enteros de 0 a $m - 1$. En parejas: complete la columna de la derecha en función de m .

```
int suma_hasta(int m) {  
    int i = 0; _____ 1  
    int suma = 0; _____ 1  
    while (i < m) {  
        suma = suma + i;  
        i = i + 1;  
    }  
    return suma; _____ 1  
}
```



Línea	Veces
int i = 0;	??
int suma = 0;	??
while (i < m)	??
suma = suma + i;	??
i = i + 1;	??
return suma;	??

Traza de suma_hasta(3) I

Una fila por cada evaluación de la condición:

Evaluación	i	$i < 3?$	Acción
1	0	cierto	suma = 0, $i = 1$
2	1	cierto	suma = 1, $i = 2$
3	2	cierto	suma = 3, $i = 3$
4	3	falso	sale del ciclo

La condición se evaluó 4 veces; el cuerpo se ejecutó 3 veces.
¿Por qué la condición se evalúa una vez más que el cuerpo?

Una vez más que el cuerpo l

La evaluación que falla también cuenta: el programa la ejecuta para descubrir que debe salir. Con entrada m , la condición entra m veces y falla 1 vez:

$$\text{evaluaciones de la condición} = m + 1.$$

El conteo completo I

Línea	Veces
<code>int i = 0;</code>	1
<code>int suma = 0;</code>	1
<code>while (i < m)</code>	$m + 1$
<code>suma = suma + i;</code>	m
<code>i = i + 1;</code>	m
<code>return suma;</code>	1

$$T(m) = 1 + 1 + (m + 1) + m + m + 1 = 3m + 4.$$

T en acción I

m	$T(m) = 3m + 4$
3	13
10	34
100	304
1000	3004

Cada vez que m se multiplica por 10, $T(m)$ se multiplica por cerca de 10: el costo crece en línea recta. Es el mismo patrón de la tabla de tiempos del inicio ($0,012 \rightarrow 0,118 \rightarrow 1,141$), pero ahora el número no depende de la máquina. Cuando T crece así, el costo es lineal.

Ahora usted I

Sumar los elementos de un arreglo. En parejas: complete el conteo, escriba $T(n)$ y compárelo con el de `suma_hasta`.

Línea		Veces
<code>int i = 0;</code>	1	??
<code>int suma = 0;</code>	1	??
<code>while (i < n)</code>		?? $n+1$
<code>suma = suma + datos[i];</code>		?? n
<code>i = i + 1;</code>		?? n
<code>return suma;</code>	1	??

$$3n + 4$$

Ahora usted: respuesta I

Línea	Veces
<code>int i = 0;</code>	1
<code>int suma = 0;</code>	1
<code>while (i < n)</code>	$n + 1$
<code>suma = suma + datos[i];</code>	n
<code>i = i + 1;</code>	n
<code>return suma;</code>	1

$$T(n) = 3n + 4.$$

El esqueleto del conteo es el mismo de `suma_hasta`: un ciclo que recorre la entrada completa produce un costo lineal.

Contenido I

- 1 ¿Cuál algoritmo es mejor?
- 2 Contar instrucciones línea a línea
- 3 Mejor y peor caso**
- 4 Cuando el conteo crece más rápido
- 5 Ejercicios
- 6 Resumen y referencias
- 7 Pistas de los ejercicios

Buscar un valor I

Decidir si el valor v aparece en el arreglo.

```
int buscar(int datos[], int n, int v) {  
    int esta = 0; _____ 1  
    int i = 0; _____ 1  
    while (i < n) { _____ 4 1  
        if (datos[i] == v) { n  
            esta = 1; K  
        }  
        i = i + 1; _____ n  
    }  
    return esta; _____ 1  
}
```

$3n + 4 + K$

En parejas: cuente las líneas que pueda. Hay una que no se deja contar con lo visto hasta ahora. ¿Cuál es, y de qué depende?

La línea rebelde I

Línea	Veces
<code>int esta = 0;</code>	1
<code>int i = 0;</code>	1
<code>while (i < n)</code>	$n + 1$
<code>if (datos[i] == v)</code>	n
<code>esta = 1;</code>	k
<code>i = i + 1;</code>	n
<code>return esta;</code>	1

La línea `esta = 1;` se ejecuta k veces, donde k es el número de apariciones de v . No depende solo de n : depende de los datos.

$$T(n) = 3n + k + 4.$$

Mejor y peor caso I

Definición

Para un tamaño de entrada n , el peor caso es la entrada que obliga a ejecutar el mayor número de instrucciones, y el mejor caso, la que exige el menor (CLRS, sección 2.2).

Para buscar:

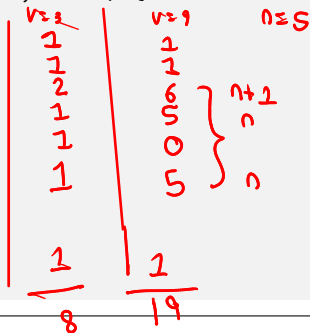
- Mejor caso: v no aparece, $k = 0$, y $T(n) = 3n + 4$.
- Peor caso: v está en todas las posiciones, $k = n$, y $T(n) = 4n + 4$.

Los dos casos son lineales: este algoritmo recorre el arreglo completo pase lo que pase.

Una variación al borde I

El mismo problema, con una condición de ciclo compuesta: el recorrido se interrumpe apenas aparece v .

```
int buscar_corte(int datos[], int n, int v) {  
    int esta = 0;  
    int i = 0;  
    while (i < n && esta == 0) {  
        if (datos[i] == v) {  
            esta = 1;  
        }  
        i = i + 1;  
    }  
    return esta;  
}
```



En parejas: trace el ciclo con $\text{datos} = \{3, 4, 5, 1, 6\}$ y $v = 3$, y después con $v = 9$. ¿Cuánto cuesta cada uno de los dos casos?

Mejor caso: v de primero l

Con $\text{datos} = \{3, 4, 5, 1, 6\}$ y $v = 3$:

Evaluación	i	$i < n?$	$esta = 0?$	Acción
1	0	cierto	cierto	$esta = 1, i = 1$
2	1	cierto	falso	sale del ciclo

El conteo: 2 inicializaciones, 2 evaluaciones de la condición, 1 if, 1 asignación, 1 incremento y 1 return:

$$T = 8.$$

Ocho instrucciones, sin importar si el arreglo tiene 5 elementos o un millón: el mejor caso es constante.

Peor caso: v no está l

El ciclo recorre todo el arreglo y esta nunca cambia:

Línea	Veces
<code>int esta = 0;</code>	1
<code>int i = 0;</code>	1
<code>while (i < n && esta == 0)</code>	$n + 1$
<code>if (datos[i] == v)</code>	n
<code>esta = 1;</code>	0
<code>i = i + 1;</code>	n
<code>return esta;</code>	1

$$T(n) = 3n + 4.$$

Peor caso: v no está II

El mismo código, dos comportamientos

`buscar_corte` es constante en el mejor caso y lineal en el peor. Cuando no se dice otra cosa, en este curso se analiza el peor caso: es la garantía que el algoritmo cumple con cualquier entrada (CLRS, sección 2.2).

Contenido I

- 1 ¿Cuál algoritmo es mejor?
- 2 Contar instrucciones línea a línea
- 3 Mejor y peor caso
- 4 Cuando el conteo crece más rápido
- 5 Ejercicios
- 6 Resumen y referencias
- 7 Pistas de los ejercicios

Un ciclo adentro de otro I

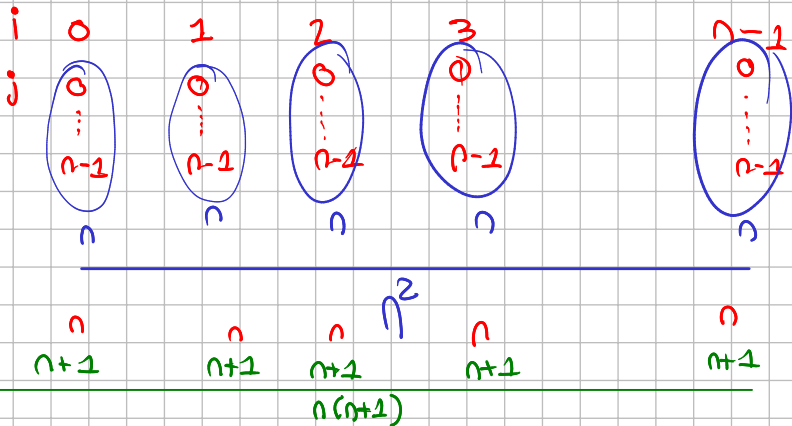
Sumar todos los productos $\text{datos}[i] * \text{datos}[j]$, con todas las parejas de posiciones (i, j) .

```
int suma_productos(int datos[], int n) {  
    int suma = 0; 1  
    int i = 0; 1  
    → while (i < n) { →  $n+1$   
        int j = 0; →  $n$   
        while (j < n) { →  $n(n+1)$   
            suma = suma + datos[i] * datos[j];  $n^2$   
            j = j + 1;  $n^2$   
        }  
        i = i + 1; →  $n$   
    }  
    return suma; 1  
}
```

```

int j = 0; → n
while (j < n) {
    suma = suma + datos[i] * datos[j];
    j = j + 1;
}

```



Un ciclo adentro de otro II

El ciclo interno completo se repite una vez por cada vuelta del externo. En parejas: complete el conteo y escriba $T(n)$.

Línea	Veces
<code>int suma = 0;</code>	??
<code>int i = 0;</code>	??
<code>while (i < n)</code>	??
<code>int j = 0;</code>	??
<code>while (j < n)</code>	??
<code>suma = suma + datos[i] * datos[j];</code>	??
<code>j = j + 1;</code>	??
<code>i = i + 1;</code>	??
<code>return suma;</code>	??

El conteo del ciclo anidado I

Línea	Veces
<code>int suma = 0;</code>	1
<code>int i = 0;</code>	1
<code>while (i < n)</code>	$n + 1$
<code>int j = 0;</code>	n
<code>while (j < n)</code>	$n(n + 1)$
<code>suma = suma + datos[i] * datos[j];</code>	n^2
<code>j = j + 1;</code>	n^2
<code>i = i + 1;</code>	n
<code>return suma;</code>	1

$$T(n) = 1 + 1 + (n + 1) + n + (n^2 + n) + n^2 + n^2 + n + 1 = 3n^2 + 4n + 4.$$

La tabla del crecimiento I

Un ciclo simple contra un ciclo anidado:

n	$3n + 4$	$3n^2 + 4n + 4$
10	34	344
100	304	30 404
1000	3004	3 004 004

Con $n = 1000$, el ciclo anidado ejecuta mil veces más instrucciones. Para n grande, el término que más crece decide todo lo demás. La notación asintótica, que le pone nombre a esta idea, es el tema de la siguiente sesión.

La pregunta del inicio I

Ya se puede responder con números. Para n par, con k valores pares en posiciones pares:

Línea	contar_v1	contar_v2
Inicializaciones	2	2
Condición del while	$n + 1$	$n/2 + 1$
if	n	$n/2$
cuenta = cuenta + 1;	k	k
Incremento de i	n	$n/2$
return	1	1

$$T_1(n) = 3n + k + 4 \quad T_2(n) = \frac{3}{2}n + k + 4.$$

La respuesta I

Con $n = 1000$ y $k = 250$:

$$T_1(1000) = 3254 \quad T_2(1000) = 1754.$$

- Las dos soluciones son lineales: al multiplicar n por 10, ambas multiplican su costo por cerca de 10.
- `contar_v2` ejecuta cerca de la mitad de las instrucciones: visita solo las posiciones que importan.

Las dos crecen al mismo ritmo; la diferencia está en la constante que las acompaña.

Contenido I

- 1 ¿Cuál algoritmo es mejor?
- 2 Contar instrucciones línea a línea
- 3 Mejor y peor caso
- 4 Cuando el conteo crece más rápido
- 5 Ejercicios**
- 6 Resumen y referencias
- 7 Pistas de los ejercicios

Ejercicio 1 I

Cuente las instrucciones de menor y escriba T . ¿Tiene mejor y peor caso?

```
int menor(int a, int b) {  
    int m = a; _____> 1  
    if (b < a) { _____> 1  
        m = b; _____K  
    }  
    return m; _____1  
}
```

3+K

$b > a$	$b < a$
3	4

Ejercicio 2 I

Dos ciclos, uno después del otro. Escriba $T(n)$. ¿El costo es lineal o cuadrático?

```
int doble_suma(int n) {  
    int i = 0; 1  
    int suma = 0; 1  
    while (i < n) {  $n+1$   
        suma = suma + i;  $n$   
        i = i + 1;  $n$   
    }  
    i = 0; 1  
    while (i < n) {  $n+1$   
        suma = suma + i * i;  $n$   
        i = i + 1;  $n$   
    }  
    return suma; 1  
}
```

$$6n + 6$$

Ejercicio 3 I

El índice avanza de tres en tres. ¿Cuántas veces itera el ciclo?
Escriba $T(n)$.

```
int saltos(int n) {  
    int i = 0;  
    int cuenta = 0;  
    while (i < n) {  
        cuenta = cuenta + 1;  
        i = i + 3;  
    }  
    return cuenta;  
}
```

$\lceil \frac{n}{3} \rceil$
 $\lceil \frac{10}{3} \rceil = 4$
 $\lceil \frac{19}{3} \rceil = 7$

$$i = n - 1$$

$\left\{ \begin{array}{l} 0, 3, 6, 9, 12 \quad n=10 \\ 0, 3, 6, 9, 12, 15, 18 \quad n=19 \\ 0, 3, 6, \dots, 30, 33 \quad n=31 \end{array} \right.$

$\frac{10}{3} = 4$
 $\frac{19}{3} = 7$
 $\frac{31}{3} = 11$

$$\lceil 3.2 \rceil = 4$$
$$\lfloor 3.9 \rfloor = 3$$

$$\lceil \frac{n}{3} \rceil$$

Ejercicio de cierre I

El ciclo interno depende del externo. Encuentre $T(n)$.

```
int triangulo(int n) {  
    int i = 0; 1  
    int cuenta = 0; 1  
    while (i < n) {  $n+1$   
        int j = 0;  $n$   
        while (j < i) {  
             $\text{cuenta} = \text{cuenta} + 1;$   
             $j = j + 1;$   
        }  
         $i = i + 1;$   $n$   
    }  
    return cuenta; 1  
}
```

i	0	1	2	3	...	n-1
j	0	1	2	3	...	n-1
	0	1	2	3	...	n-1

$0 + 1 + 2 + 3 + \dots + n$

$\sum_{j=0}^n i = \frac{n(n+1)}{2}$

Contenido I

- 1 ¿Cuál algoritmo es mejor?
- 2 Contar instrucciones línea a línea
- 3 Mejor y peor caso
- 4 Cuando el conteo crece más rápido
- 5 Ejercicios
- 6 Resumen y referencias
- 7 Pistas de los ejercicios

Lo que queda de la sesión

- El reloj mide corridas: cambia con la máquina, la carga y el compilador. Contar instrucciones mide el algoritmo.
- El conteo línea a línea produce una función T del tamaño de la entrada.
- La condición de un `while` se evalúa una vez más que su cuerpo.
- Cuando el conteo depende de los datos, el mejor y el peor caso lo acotan; se reporta el peor.
- Dos ciclos seguidos suman su costo; un ciclo adentro de otro lo multiplica.

Referencias I



T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein.
Introduction to Algorithms. 4.^a ed., MIT Press, 2022. Sección
2.2 (*Analyzing algorithms*).



R. Thareja. *Data Structures Using C*. Oxford University Press,
2018. Capítulo 2.

Contenido I

- 1 ¿Cuál algoritmo es mejor?
- 2 Contar instrucciones línea a línea
- 3 Mejor y peor caso
- 4 Cuando el conteo crece más rápido
- 5 Ejercicios
- 6 Resumen y referencias
- 7 Pistas de los ejercicios

Pistas I

- **Ejercicio 1:** no hay ciclos, así que el costo es constante. El `if` hace que la asignación $m = b$; se ejecute 0 o 1 veces: ahí están el mejor y el peor caso.
- **Ejercicio 2:** cuente cada ciclo por separado y sume los totales. Dos recorridos completos siguen siendo un costo lineal; lo cuadrático aparece al anidar, no al encadenar.
- **Ejercicio 3:** la condición deja de cumplirse cuando i alcanza a n saltando de tres en tres; el ciclo itera $\lceil n/3 \rceil$ veces. El costo sigue siendo lineal, con una constante menor.

El ejercicio de cierre se resuelve con lo visto en la sesión; como ayuda, la traza con $n = 4$ muestra el patrón del ciclo interno.